UNIVERSIDAD NACIONAL AUTÓNOMA DE NICARAGUA

UNAN – LEÓN



ÁREA DE CONOCIMIENTO DE CIENCIAS Y TECNOLOGÍA

Carrera: Ing. En Telemática.

Componente: Proyecto Integrador

Tema: Avance del proyecto

Integrantes: Ángel Danilo Álvarez Padilla

Isabella Aracely Arosteguí Rojas

José Carlos Solano Duarte

Docente: Ing. Ernesto Espinoza

Fecha: Domingo, 13 de septiembre de 2026

"47/19: A la libertad, por la universidad"

Contenido

I. Introducción....1
1.1 Propósito....1
1.2 Alcance del producto....1
1.3 Definiciones y acrónimos....1
II. Alcance del sistema....3
2.1 Funcionalidades principales....3
2.2 Funcionalidades fuera de alcance de la primera versión....3
III. Principales procesos del sistema....4
3.1 Proceso de registro de usuario....4
3.2 Proceso de inicio de sesión....4
3.3 Proceso de depósito....4
3.4 Proceso de retiro....4
3.5 Proceso de juegos y actualización del saldo....4
3.5.1 Motor de tragaperras....5
3.5.2 Motor de blackjack....5
3.5.3 Motor de dados....5
3.5.4 Motor de carrera de caballos....5
3.5.5 Motor de lotería....5
3.6 Proceso de perfil, historial y tabla de líderes....5
3.7 Proceso transversal de persistencia....5
IV. Planificación del proyecto....6
V. Anexos....7
Pseudocódigo del programa + código en python....7
Matriz De Interdependencias....10
Diagrama de diseño Top-Down....13

I. Introducción

Este documento presenta la especificación inicial del Sistema de Casino Virtual, una aplicación de consola diseñada para simular un entorno de juegos de azar. El sistema permite registrar usuarios, iniciar sesión, administrar el saldo mediante depósitos y retiros, participar en distintos juegos y consultar resultados, historial y récords personales.

La propuesta se organiza mediante la metodología Top Down: primero se define el problema general y después se descompone en módulos funcionales cada vez más específicos. Esta organización facilita comprender cómo los datos capturados por el sistema se transforman en resultados y cómo dichos resultados se conservan para futuras consultas.

La solución se divide en tres bloques funcionales. El módulo de entrada captura usuarios, credenciales, transacciones y apuestas; el módulo de procesamiento ejecuta los motores de juego, actualiza el saldo y calcula los récords; y el módulo de salida presenta los resultados, genera reportes básicos y conserva la información en archivos JSON.

1.1 Propósito

El propósito del sistema es ofrecer una aplicación de consola que simule un casino virtual y que permita administrar de manera organizada la información de los usuarios, sus saldos, las apuestas, los resultados de las partidas y sus estadísticas. El sistema debe entregar confirmaciones claras, actualizar el saldo después de cada operación y mantener los registros disponibles para futuras consultas.

1.2 Alcance del producto

El producto se plantea como una aplicación de consola con persistencia en archivos JSON. Su alcance comprende el registro y autenticación de usuarios, la administración básica del saldo, cinco juegos de azar, la generación de historial de transacciones, la consulta del perfil y la visualización de una tabla global de líderes.

La primera versión se enfoca en la lógica de negocio y en la modularidad del programa. Por ello, la interfaz se limita a menús de texto y la información se almacena mediante archivos JSON, de acuerdo con la descomposición propuesta en el documento base.

1.3 Definiciones y acrónimos

Término	Definición
Top Down	Metodología que parte del problema general y lo divide progresivamente en

Término	Definición
	módulos y funciones más pequeñas.
SRS	Software Requirements Specification o especificación de requisitos de software.
JSON	Formato de texto utilizado para almacenar datos estructurados en archivos.
PSeInt	Herramienta educativa para escribir y ejecutar algoritmos en pseudocódigo.
Motor de juego	Módulo que ejecuta la lógica de un juego, determina el resultado y calcula el pago correspondiente.
Persistencia	Conservación de la información para que pueda recuperarse después de cerrar el programa.

II. Alcance del sistema

El sistema administrará la interacción básica de los usuarios con un casino virtual en modo consola. Todas las operaciones relacionadas con el saldo y las partidas estarán vinculadas al usuario autenticado y deberán generar la información necesaria para actualizar estadísticas e historial.

2.1 Funcionalidades principales

- Registro de usuarios mediante nombre de usuario y contraseña, verificando que el nombre no esté registrado.
- Inicio de sesión mediante validación de las credenciales almacenadas.
- Depósitos y retiros con validación del monto y del saldo disponible.
- Gestión de apuestas asociadas al usuario autenticado.
- Simulación de tragaperras, blackjack, dados, carrera de caballos y lotería.
- Actualización del saldo después de cada transacción o partida.
- Cálculo de récords personales: racha de victorias, mayor ganancia, total apostado y juego favorito.
- Registro y consulta del historial de transacciones.
- Visualización de una tabla global de líderes por saldo o racha.
- Carga y guardado de usuarios, transacciones y récords mediante archivos JSON.

2.2 Funcionalidades fuera de alcance de la primera versión

- Interfaz gráfica, aplicación web o aplicación móvil; la primera versión funcionará en consola.
- Uso de dinero real, conexión con bancos, billeteras digitales o pasarelas de pago.
- Base de datos relacional; la persistencia inicial se realizará en archivos JSON.
- Partidas multijugador en línea y conexión con servidores externos.
- Sistemas avanzados de administración, auditoría o reportes empresariales que no aparecen en la definición inicial.

III. Principales procesos del sistema

Los procesos siguientes describen el comportamiento principal de la aplicación desde la perspectiva del usuario y del sistema. Cada proceso corresponde a los módulos y submódulos definidos en la metodología Top Down, e indica las validaciones y resultados esperados.

3.1 Proceso de registro de usuario

1. El usuario selecciona la opción Registrar usuario.
2. El sistema solicita el nombre de usuario y la contraseña.
3. El sistema revisa los usuarios almacenados para comprobar que el nombre no exista.
4. Si el nombre ya está registrado, el sistema muestra un mensaje y solicita otros datos.
5. Si los datos son válidos, el sistema crea el usuario con saldo inicial igual a cero.
6. El sistema inicializa los contadores de partidas, victorias, derrotas, racha, mayor ganancia y juego favorito.
7. El sistema guarda la información y confirma que el registro fue exitoso.

3.2 Proceso de inicio de sesión

8. El usuario selecciona la opción Iniciar sesión.
9. El sistema solicita el nombre de usuario y la contraseña.
10. El sistema compara las credenciales con los datos guardados.
11. Si las credenciales no coinciden, el sistema informa que el acceso no es válido.
12. Si coinciden, el sistema identifica al usuario y habilita las operaciones de depósitos, retiros, juegos y consultas.

3.3 Proceso de depósito

13. El usuario inicia sesión y selecciona Depositar.
14. El sistema solicita el monto del depósito.
15. El sistema valida que el monto sea mayor que cero.
16. Si el monto es válido, el sistema lo suma al saldo actual.
17. El sistema registra la operación como una transacción de tipo depósito.
18. El sistema guarda los datos y muestra el nuevo saldo.

3.4 Proceso de retiro

19. El usuario inicia sesión y selecciona Retirar.
20. El sistema solicita el monto del retiro.
21. El sistema valida que el monto sea mayor que cero y que no supere el saldo disponible.
22. Si la validación es correcta, el sistema resta el monto del saldo.
23. El sistema registra la operación como una transacción de tipo retiro.
24. El sistema confirma la operación y muestra el saldo actualizado.

3.5 Proceso de juegos y actualización del saldo

25. El usuario autenticado selecciona uno de los cinco juegos disponibles.
26. El sistema solicita el monto de la apuesta y las decisiones específicas del juego.
27. El sistema valida que la apuesta sea mayor que cero y que exista saldo suficiente.
28. El motor correspondiente ejecuta la lógica del juego y determina si el usuario ganó, perdió o empató.

29. El sistema descuenta la apuesta y, cuando corresponde, suma el premio al saldo.
30. El sistema registra el resultado, actualiza la racha, la mayor ganancia y el juego favorito.
31. El sistema muestra el resultado de la partida y el saldo actualizado.

3.5.1 Motor de tragaperras

Genera una combinación aleatoria de símbolos y calcula el pago con base en la tabla de premios definida para el juego. El resultado se envía al control de saldo, al historial y al cálculo de récords.

3.5.2 Motor de blackjack

Reparte las cartas o valores iniciales, permite al usuario pedir carta o plantarse y hace que la casa pida mientras su puntaje sea menor que 17. Luego compara los puntajes y determina el ganador.

3.5.3 Motor de dados

Lanza los dados y evalúa la opción elegida por el usuario: par o impar, número exacto, mayor que siete o menor que siete.

3.5.4 Motor de carrera de caballos

Simula el avance aleatorio de varios caballos y compara el ganador con el caballo seleccionado por el usuario para calcular el resultado de la apuesta.

3.5.5 Motor de lotería

Sortea un número y calcula el pago según la coincidencia exacta o la cercanía entre el número elegido y el número sorteado.

3.6 Proceso de perfil, historial y tabla de líderes

32. El usuario autenticado selecciona Perfil para consultar su saldo, racha, mayor ganancia, total apostado y juego favorito.
33. El usuario selecciona Historial para visualizar las transacciones y resultados registrados.
34. Cuando se selecciona Tabla de líderes, el sistema ordena o compara los usuarios según el saldo o la racha y muestra los primeros lugares.
35. La información mostrada se obtiene de los datos actualizados por los módulos de saldo, récords e historial.

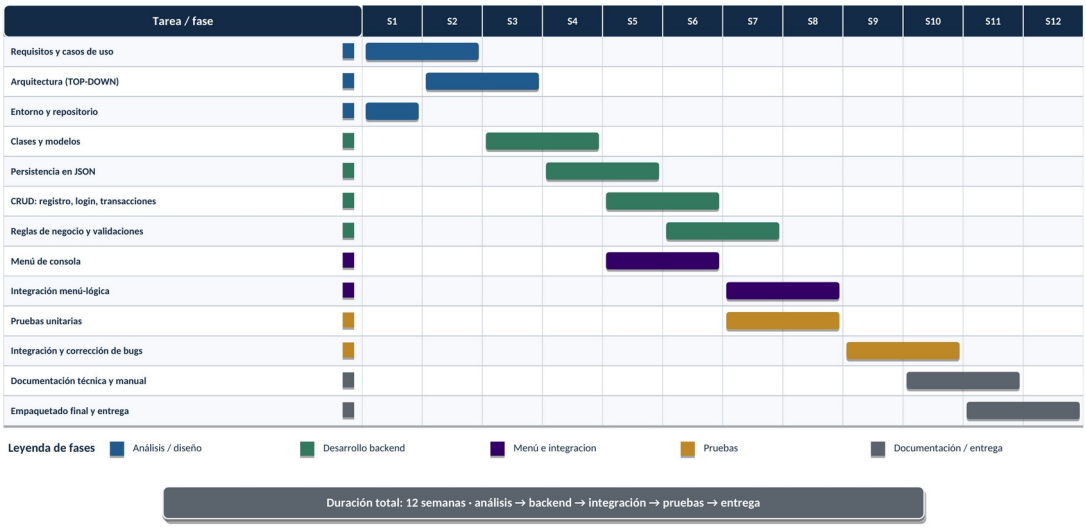
3.7 Proceso transversal de persistencia

36. Al iniciar una operación, el sistema carga los datos necesarios desde los archivos JSON.
37. Cada registro, depósito, retiro o partida modifica los datos en memoria.
38. El sistema registra la transacción y actualiza las estadísticas relacionadas.
39. El sistema guarda los usuarios, transacciones y récords para conservar la información después de cerrar la aplicación.

IV. Planificación del proyecto

Modelo de Gantt del proyecto

Planificación por semanas para el desarrollo modular del Sistema de Casino Virtual



Duración total: 12 semanas · análisis → backend → integración → pruebas → entrega

V. Anexos

Pseudocódigo del programa + código en python

El siguiente pseudocódigo integra el flujo principal del sistema en un único algoritmo. Para facilitar su ejecución académica, los datos se mantienen en arreglos durante la ejecución; la persistencia en JSON queda definida para la implementación posterior en el lenguaje de programación del proyecto.

```
Módulo A.1.1. registrar_usuario():
    Imprimir "Ingrese nombre de usuario:"
    Leer usuario
    Si usuario existe en usuarios_bd Entonces
        Imprimir "Error: El usuario ya existe."
    Sino
        Imprimir "Ingrese contraseña:"
        Leer contrasena
        Agregar usuario a usuarios_bd con contrasena y saldo
inicial (ej. 0.0)
        Imprimir "Cuenta creada exitosamente."


Módulo A.1.2. iniciar_sesion():
    Imprimir "Ingrese nombre de usuario:"
    Leer usuario
    Imprimir "Ingrese contraseña:"
    Leer contrasena
    Si usuario existe en usuarios_bd Y
usuarios_bd[usuario].contrasena == contrasena Entonces
        Imprimir "Inicio de sesión exitoso."
        Retornar usuario
    Sino
        Imprimir "Credenciales incorrectas."
        Retornar Nulo
```

```
Módulo A.2.1. depositar(usuario_actual):
    Imprimir "Ingrese el monto a depositar:"
    Leer monto
    Si monto > 0 Entonces
        usuarios_bd[usuario_actual].saldo = 
usuarios_bd[usuario_actual].saldo + monto
        Imprimir "Depósito exitoso. Nuevo saldo:", 
usuarios_bd[usuario_actual].saldo
    Sino
        Imprimir "Error: El monto debe ser mayor a cero."


Módulo A.3.1. jugar_tragaperras(usuario_actual):
    Imprimir "Ingrese el monto de apuesta:"
    Leer apuesta
    Si apuesta > 0 Y apuesta <= usuarios_bd[usuario_actual].saldo 
Entonces
        Imprimir "Apuesta aceptada por el monto:", apuesta
        // La ejecución de la máquina tragaperras se insertará 
aquí
    Sino
        Imprimir "Error: Saldo insuficiente o monto inválido."
# Simulación de base de datos en memoria (diccionario)
usuarios_bd = {}


def registrar_usuario():
    usuario = input("Ingrese nombre de usuario: ")


    if usuario in usuarios_bd:
        print("Error: El usuario ya existe. Intente con otro 
nombre.")
    else:
```

```
        contrasena = input("Ingrese contraseña: ")
        # Se crea la cuenta con un saldo inicial de 0.0
        usuarios_bd[usuario] = {
            "contrasena": contrasena,
            "saldo": 0.0
        }
        print("Cuenta creada exitosamente.")

def iniciar_sesion():
    usuario = input("Ingrese nombre de usuario: ")
    contrasena = input("Ingrese contraseña: ")

    # Verificación de credenciales
    if usuario in usuarios_bd and usuarios_bd[usuario]
["contrasena"] == contrasena:
        print("Inicio de sesión exitoso.")
        return usuario
    else:
        print("Error: Usuario o contraseña incorrectos.")
        return None

def depositar(usuario_actual):
    try:
        monto = float(input("Ingrese el monto a depositar: "))
        if monto > 0:
            usuarios_bd[usuario_actual]["saldo"] += monto
            nuevo_saldo = usuarios_bd[usuario_actual]["saldo"]
            print(f"Depósito exitoso. Su nuevo saldo es: $
{nuevo_saldo:.2f}")
        else:
```

```
            print("Error: El monto a depositar debe ser mayor a
0.")
    except ValueError:
        print("Error: Por favor ingrese un valor numérico
válido.")


def jugar_tragaperras(usuario_actual):
    try:
        apuesta = float(input("Ingrese el monto de su apuesta: "))
        saldo_actual = usuarios_bd[usuario_actual]["saldo"]
        if apuesta <= 0:
            print("Error: La apuesta debe ser mayor a 0.")
        elif apuesta > saldo_actual:
            print(f"Error: Saldo insuficiente. Su saldo actual es:
${saldo_actual:.2f}")
        else:
            print(f"Apuesta de ${apuesta:.2f} registrada. ¡Listos
para jugar!")
            # Aquí se integrará posteriormente la mecánica de los
rodillos
    except ValueError:
        print("Error: Por favor ingrese un valor numérico válido
para la apuesta.")
```

Matriz De Interdependencias

La matriz identifica qué datos recibe cada módulo, qué resultado produce y qué otros módulos utilizan ese resultado. Se transcribe y organiza a partir de la matriz completa proporcionada.

Observación: los cinco motores de juego comparten el mismo patrón de salida hacia el control de saldo, el registro de transacciones y el cálculo de récords; lo que cambia es la lógica interna de cada juego.

Nombre del módulo	Módulo superior	Código	Datos de entrada	Datos de salida	Interdependencias
Registrar usuario	Gestión de Registro/Login	A.1.1	usuario, contraseña	usuario_id, saldo inicial	→ B.2.1 (crea saldo inicial)
Iniciar sesión	Gestión de Registro/Login	A.1.2	usuario, contraseña	usuario_id autenticado	→ A.2, A.3 (habilita acceso a transacciones y juegos)
Depositar	Depósitos y Retiros	A.2.1	usuario_id, monto	nuevo saldo	→ B.2.1 actualizar_saldo() → C.1.1 registrar_transaccion()
Retirar	Depósitos y Retiros	A.2.2	usuario_id, monto	nuevo saldo	→ B.2.1 actualizar_saldo() → C.1.1 registrar_transaccion()
Jugar tragaperras	Gestión de Apuestas	A.3.1	usuario_id, apuesta	apuesta validada	→ B.1.1 motor_tragaperras()
Jugar blackjack	Gestión de Apuestas	A.3.2	usuario_id, apuesta, decisiones	apuesta y decisiones validadas	→ B.1.2 motor_blackjack()
Jugar dados	Gestión de Apuestas	A.3.3	usuario_id, apuesta, tipo_apuesta	apuesta validada	→ B.1.3 motor_dados()
Jugar carrera	Gestión de Apuestas	A.3.4	usuario_id, apuesta, caballo_elegido	apuesta validada	→ B.1.4 motor_carrera()
Jugar lotería	Gestión de Apuestas	A.3.5	usuario_id, apuesta, número_elegido	apuesta validada	→ B.1.5 motor_loteria()
Motor tragaperras	Motores de Juego	B.1.1	apuesta	resultado, monto_pago	→ B.2.1, C.1.1, B.3.1/B.3.2
Motor blackjack	Motores de Juego	B.1.2	apuesta, decisiones	resultado, monto_pago	→ B.2.1, C.1.1, B.3.1/B.3.2
Motor dados	Motores de Juego	B.1.3	apuesta, tipo_apuesta	resultado, monto_pago	→ B.2.1, C.1.1, B.3.1/B.3.2
Motor carrera	Motores de Juego	B.1.4	apuesta, caballo_elegido	resultado, monto_pago	→ B.2.1, C.1.1, B.3.1/B.3.2
Motor lotería	Motores de Juego	B.1.5	apuesta, número_elegido	resultado, monto_pago	→ B.2.1, C.1.1, B.3.1/B.3.2
Actualizar saldo	Control de Saldo	B.2.1	usuario_id, monto, operación	saldo actualizado	→ C.2.1, A.3.x (verificación previa)
Actualizar racha	Cálculo de Récords	B.3.1	usuario_id, resultado	racha actualizada	→ C.2.1, C.2.2
Actualizar mayor ganancia	Cálculo de Récords	B.3.2	usuario_id, monto	récord actualizado	→ C.2.1
Calcular juego favorito	Cálculo de Récords	B.3.3	usuario_id, historial de partidas	juego más jugado	→ C.2.1

Nombre del módulo	Módulo superior	Código	Datos de entrada	Datos de salida	Interdependencias
Registrar transacción	Generador de Historial	C.1.1	usuario_id, tipo, monto	registro con fecha/hora	→ C.1.2, C.3.1 (guardar)
Imprimir historial	Generador de Historial	C.1.2	usuario_id	lista de transacciones en pantalla	módulo terminal
Imprimir perfil	Generador de Récords	C.2.1	usuario_id	saldo, racha, récords en pantalla	módulo terminal
Imprimir tabla de líderes	Generador de Récords	C.2.2	lista de usuarios	ranking en pantalla	módulo terminal
Cargar / Guardar JSON	Persistencia	C.3.1	nombre de archivo, datos	datos leídos o confirmación de guardado	← usado por A.1, B.2.1, C.1.1 (punto único de escritura)

Diagrama de diseño Top-Down

